

USAC – CUNOC

División de Ciencias de la Ingeniería

MANUAL TÉCNICO

Sistema de Generación de Horarios

Algoritmo Genético para Optimización de Horarios Académicos

Proyecto
Genetic Scheduler – CUNOC
Versión
1.0.0
Tecnologías
Vue 3 · Node.js / Express · PostgreSQL
Fecha
Marzo 2026

1. Introducción

1.1 Propósito del Sistema

El Sistema de Generación de Horarios es una aplicación web desarrollada para la División de Ciencias de la Ingeniería del Centro Universitario de Occidente (CUNOC) de la Universidad de San Carlos de Guatemala (USAC). Su objetivo principal es automatizar y optimizar la asignación de horarios académicos mediante la implementación de un Algoritmo Genético (AG).

El sistema resuelve el problema de asignación de horarios académicos — un problema NP-completo — encontrando soluciones de alta calidad que satisfacen múltiples restricciones simultáneamente: disponibilidad de docentes, capacidad de salones, ausencia de conflictos de tiempo y requisitos por jornada.

1.2 Alcance

El sistema cubre las siguientes funcionalidades principales:

- Gestión de catálogos: carreras, cursos, laboratorios, docentes, salones, secciones.
- Importación masiva de datos mediante archivos CSV.
- Configuración paramétrica del algoritmo genético.
- Ejecución del algoritmo genético y monitoreo en tiempo real.
- Visualización, edición manual y exportación de horarios generados.
- Reportes de conflictos, estadísticas y desempeño del algoritmo.
- Herramientas de mantenimiento y reset de base de datos.

1.3 Stack Tecnológico

Capa	Tecnología	Versión	Propósito
Frontend	Vue 3	3.x	Interfaz de usuario reactiva
Frontend	Pinia	2.x	Gestión de estado global
Frontend	Vue Router	4.x	Enrutamiento SPA
Backend	Node.js	18+	Runtime del servidor
Backend	Express	4.x	Framework HTTP / REST API
Backend	pg (node-postgres)	8.x	Cliente PostgreSQL
Backend	multer	1.x	Carga de archivos CSV
Backend	csv-parse	6.x	Parseo de archivos CSV
Backend	ws	8.x	WebSocket para progreso AG
Base Datos	PostgreSQL	14+	Persistencia relacional
DevOps	nodemon	3.x	Reinicio automático en dev

Capa	Tecnología	Versión	Propósito
DevOps	dotenv	16.x	Variables de entorno

2. Arquitectura del Sistema

2.1 Visión General

El sistema sigue una arquitectura cliente-servidor de tres capas:

Capa	Descripción
Presentación (Frontend)	SPA construida con Vue 3. Consume la REST API del backend mediante fetch/axios. Renderiza vistas reactivas para cada módulo del sistema.
Lógica de Negocio (Backend)	API REST en Express. Orquesta los servicios, contiene el motor del algoritmo genético y expone todos los endpoints.
Datos (PostgreSQL)	Base de datos relacional que persiste catálogos, horarios generados, historial de evolución y configuración del AG.

2.2 Estructura de Directorios — Backend

```
api/
├─ index.js           # Punto de entrada – Express app
├─ package.json
├─ .env               # Variables de entorno (DB, PORT)
├─ src/
│   ├─ db/            # Conexión PostgreSQL (pool)
│   ├─ routes/        # Router principal + sub-routers
│   │   ├─ index.js
│   │   ├─ carreras/  # GET / POST / PUT / DELETE
│   │   ├─ cursos/
│   │   ├─ docentes/
│   │   ├─ salones/
│   │   ├─ laboratorios/
│   │   ├─ secciones/
│   │   ├─ seccion-laboratorio/
│   │   ├─ docente-curso/
│   │   ├─ periodos/
│   │   ├─ dias-horario/
│   │   ├─ configuracion/
│   │   ├─ importaciones/
│   │   ├─ algoritmo/
│   │   ├─ horarios/
│   │   └─ reset/
│   ├─ services/      # Lógica de negocio por módulo
│   │   ├─ common/errors.js # ApiError + mapPgError
│   │   └─ ...
└─ genetico/         # Motor del Algoritmo Genético
```

```

├─ index.js
├─ contexto.js          # Carga datos de BD
├─ cromosoma.js        # Generación de individuos
├─ aptitud.js          # Función de aptitud
├─ seleccion.js        # Torneo / Ruleta
├─ cruce.js            # Un punto / Multipunto
├─ mutacion.js         # Intercambio / Reinserción
└─ algoritmo.js        # Loop principal + persistencia

```

2.3 Estructura de Directorios — Frontend

```

frontend/
├─ src/
│  ├─ main.js          # Bootstrap de la app
│  ├─ App.vue          # Componente raíz
│  ├─ router/index.js  # Vue Router
│  ├─ stores/          # Pinia stores
│  └─ views/           # Páginas/vistas
│     ├─ HomeView.vue
│     ├─ CarrerasView.vue
│     ├─ CursosView.vue
│     ├─ LaboratoriosView.vue
│     ├─ DocentesView.vue
│     ├─ SalonesView.vue
│     ├─ SeccionesView.vue
│     ├─ SeccionesLaboratorioView.vue
│     ├─ DocenteCursoView.vue
│     ├─ DiasHorarioView.vue
│     ├─ PeriodosView.vue
│     ├─ ImportacionesView.vue
│     ├─ ConfiguracionAgenteView.vue
│     ├─ GeneracionView.vue
│     ├─ ReportesView.vue
│     ├─ MantenimientoView.vue
│     └─ AcercaDeView.vue
└─ components/         # Componentes reutilizables
   └─ Navbar.vue
└─ public/

```

3. Base de Datos

3.1 Motor y Configuración

El sistema utiliza PostgreSQL como motor de base de datos relacional. La conexión se gestiona mediante un pool de conexiones con la librería pg (node-postgres). Las credenciales se configuran mediante variables de entorno (.env).

Variables requeridas: DATABASE_URL o DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME

3.2 Tablas del Sistema

Tabla	Descripción	Filas clave
carreras	Catálogo de carreras académicas	id, nombre, codigo
dias	Días de la semana (lunes–viernes)	id, nombre
dias_horario	Agrupaciones de días (LMV / MarJue)	id, nombre, es_laboratorio
dias_horario_dia	Relación días_horario ↔ dias	dias_horario_id, dia_id
periodos	Franjas de tiempo generadas automáticamente	id, numero, hora_inicio, hora_fin, es_manana, es_tarde
salones	Aulas y laboratorios disponibles	id, nombre, capacidad, es_laboratorio, lab_habilitado_teorico
docentes	Profesores activos del sistema	id, nombre, registro_personal, hora_entrada, hora_salida
carreras	Programas académicos	id, nombre, codigo
cursos	Cursos con tipo y configuración	id, nombre, codigo, carrera_id, semestre, tipo, tiene_laboratorio
laboratorios	Laboratorios asociados a cursos	id, curso_id, num_periodos, puede_manana, puede_tarde
secciones	Secciones por curso con opciones fijas	id, curso_id, letra, salon_fijo_id, docente_fijo_id
seccion_laboratorio	Laboratorio asignado a cada sección	id, seccion_id, laboratorio_id, salon_fijo_id
docente_curso	Qué docentes pueden impartir qué cursos	id, docente_id, curso_id, puede_laboratorio
configuracion_algoritmo	Parámetros del AG (una sola fila)	tamano_poblacion, max_generaciones, tasa_mutacion, métodos
horarios	Cabecera de cada horario generado	id, nombre, aptitud_final,

Tabla	Descripción	Filas clave
		generaciones_ejecutadas, es_activo
horario_detalle	Asignaciones individuales del horario	horario_id, seccion_id, salon_id, docente_id, dia_horario_id, periodo_inicio_id
horario_historial	Evolución de aptitud por generación	horario_id, generacion, mejor_aptitud, conflictos

3.3 Relaciones Principales

El diagrama lógico sigue estas relaciones de integridad referencial:

- cursos → carreras (carrera_id)
- laboratorios → cursos (curso_id, UNIQUE — un lab por curso)
- secciones → cursos, salones, docentes, periodos, dias_horario
- seccion_laboratorio → secciones, laboratorios, salones, docentes
- docente_curso → docentes, cursos
- horario_detalle → horarios, secciones, seccion_laboratorio, salones, docentes, dias_horario, periodos
- horario_historial → horarios (ON DELETE CASCADE)

3.4 Generación Automática de Periodos

Los periodos de clase no se ingresan manualmente. El sistema los genera automáticamente a partir de los parámetros de configuración: hora de inicio/fin de mañana y tarde, y la duración de cada periodo en minutos.

Cuando se actualiza cualquiera de estos parámetros (hora_inicio_manana, hora_fin_manana, hora_inicio_tarde, hora_fin_tarde, duracion_periodo), el servicio regenerarPeriodos.service.js elimina todos los periodos existentes y recalcula la secuencia completa.

4. Motor del Algoritmo Genético

4.1 Descripción General

El núcleo del sistema es un Algoritmo Genético (AG) implementado completamente en JavaScript (Node.js). El AG resuelve el problema de asignación de horarios como un problema de optimización combinatoria, evolucionando una población de soluciones candidatas hacia soluciones de mayor calidad.

4.2 Representación del Cromosoma

Cada individuo (cromosoma) representa un horario completo. Sus genes son un arreglo donde cada elemento corresponde a una sección teórica o una sección de laboratorio:

Atributo del Gen	Tipo	Descripción
seccion_id / seccion_lab_id	INT	Identificador de la sección (teórica o laboratorio)
salon_id	INT	Salón asignado
docente_id	INT	Docente asignado
periodo_inicio_id	INT	Periodo de inicio
periodo_fin_id	INT	Periodo de fin
dia_horario_id	INT	Agrupación de días (LMV=1, MarJue=2)
distribucion_lab	Object	Solo para labs: {martes: {...}, jueves: {...}}
es_laboratorio	Boolean	Distingue secciones teóricas de laboratorios

Nota sobre laboratorios: Los laboratorios se asignan en días Martes/Jueves con distribución flexible. Pueden acomodar 3 periodos en un día o dividirlos entre martes y jueves (3+0, 0+3, 2+1, 1+2).

4.3 Función de Aptitud

La aptitud de un individuo se calcula sumando penalizaciones (valores negativos) y bonos (valores positivos). Una aptitud perfecta (sin conflictos) sería 0 más los bonos acumulados.

Tipo	Descripción	Valor
PENALIZACIÓN	Conflicto de docente (misma persona en dos lugares)	-100 por conflicto
PENALIZACIÓN	Conflicto de salón (dos cursos en el mismo salón)	-100 por conflicto
PENALIZACIÓN	Conflicto de semestre (dos obligatorios a la vez)	-80 por conflicto
PENALIZACIÓN	Horario incorrecto (mañana/tarde no permitido)	-70 por gen

Tipo	Descripción	Valor
ÓN		
PENALIZACIÓN	Salón inadecuado (lab en salón teórico o viceversa)	-60 por gen
PENALIZACIÓN	Docente fuera de su horario laboral	-50 por gen
PENALIZACIÓN	Capacidad del salón superada	-20 por gen
BONO	Cursos del mismo semestre en bloques consecutivos	+30 por par
BONO	Salón con capacidad adecuada para el grupo	+10 por gen

4.4 Métodos de Selección

Método	ID	Descripción
Torneo	torneo	Selecciona K individuos al azar (K=5 por defecto) y devuelve el de mayor aptitud. Funciona bien con aptitudes negativas sin ajuste.
Ruleta	ruleta	Normaliza las aptitudes desplazando el mínimo a +1, luego aplica selección proporcional. Los individuos con mayor aptitud tienen mayor probabilidad de ser elegidos.

4.5 Métodos de Cruce

Método	ID	Descripción
Un Punto	un_punto	Elige un punto de corte aleatorio. El hijo 1 toma genes del padre A hasta el corte y del padre B desde el corte. El hijo 2 es el complemento.
Multipunto	multipunto	Elige dos puntos de corte. El segmento del medio se intercambia entre padres. Permite mayor variabilidad genética.

Importante: El cruce solo transfiere atributos de asignación (salon_id, docente_id, periodo_inicio_id, dia_horario_id). Los metadatos de contexto de cada gen (semestre, carrera_id, tipo, etc.) se preservan siempre del gen original.

4.6 Métodos de Mutación

Método	ID	Descripción
Intercambio	intercambio	Elige dos genes al azar del mismo tipo (cursos o labs) e intercambia sus atributos de asignación. Conservadora — reordena lo existente.

Método	ID	Descripción
Reinserción	reiserccion	Itera cada gen con probabilidad = tasa_mutacion. Reasigna docente, periodo, distribución de lab y salón con valores nuevos válidos. Más agresiva — explora nuevas zonas del espacio.

4.7 Elitismo y Criterios de Parada

El AG implementa elitismo con el top 10% de la población: estos individuos pasan sin modificación a la siguiente generación, garantizando que la mejor solución no se pierda.

Criterios de parada (el primero que se cumpla detiene la ejecución):

- Se alcanza el número máximo de generaciones (max_generaciones).
- La aptitud del mejor individuo supera la aptitud objetivo (aptitud_objetivo), si está configurada.

4.8 Flujo del Loop Principal

1. Generar población inicial aleatoria
2. Evaluar aptitud de toda la población
3. Por cada generación:
 - a. Conservar élite (top 10%) sin modificar
 - b. Seleccionar pares de padres
 - c. Cruzar padres → 2 hijos
 - d. Mutar hijos (por gen, con tasa configurable)
 - e. Evaluar nueva población y ordenar
 - f. Registrar mejor aptitud + conflictos en historial
 - g. Emitir progreso (WebSocket si hay cliente conectado)
4. Al terminar: guardar horario + historial en PostgreSQL

5. API REST — Referencia de Endpoints

Todos los endpoints tienen el prefijo base: /api

Base URL: http://localhost:3000/api

5.1 Carreras — /api/carreras

Método	Ruta	Descripción
GET	/api/carreras	Listar todas las carreras
GET	/api/carreras/:id	Obtener carrera por ID
GET	/api/carreras/codigo/:codigo	Obtener carrera por código
POST	/api/carreras	Crear nueva carrera
PUT	/api/carreras/:id	Actualizar carrera
DELETE	/api/carreras/:id	Eliminar carrera

5.2 Cursos — /api/cursos

Método	Ruta	Descripción
GET	/api/cursos	Listar todos los cursos
GET	/api/cursos/:id	Obtener curso por ID
POST	/api/cursos	Crear nuevo curso
PUT	/api/cursos/:id	Actualizar curso
DELETE	/api/cursos/:id	Eliminar curso
PATCH	/api/cursos/desactivar-pares	Desactivar cursos de semestres pares + sus labs
PATCH	/api/cursos/desactivar-impares	Desactivar cursos de semestres impares + sus labs
PATCH	/api/cursos/desactivar-todos	Desactivar todos los cursos y laboratorios
PATCH	/api/cursos/activar-todos	Activar todos los cursos y laboratorios

5.3 Docentes — /api/docentes

Método	Ruta	Body requerido	Descripción
GET	/api/docentes	—	Listar docentes

Método	Ruta	Body requerido	Descripción
GET	/api/docentes/:id	—	Obtener docente por ID
POST	/api/docentes	nombre, registro_personal, hora_entrada, hora_salida	Crear docente
PUT	/api/docentes/:id	nombre, registro_personal, hora_entrada, hora_salida	Actualizar docente
DELETE	/api/docentes/:id	—	Eliminar docente

5.4 Salones — /api/salones

Método	Ruta	Body requerido	Descripción
GET	/api/salones	—	Listar salones
GET	/api/salones/:id	—	Obtener por ID
POST	/api/salones	nombre, capacidad, es_laboratorio, disponible_manana, disponible_tarde	Crear salón
PUT	/api/salones/:id	misimos campos	Actualizar salón
DELETE	/api/salones/:id	—	Eliminar salón

5.5 Laboratorios — /api/laboratorios

Método	Ruta	Descripción
GET	/api/laboratorios	Listar laboratorios
GET	/api/laboratorios/:id	Obtener por ID
GET	/api/laboratorios/curso/:cursoId	Laboratorios de un curso
POST	/api/laboratorios	Crear laboratorio (activa tiene_laboratorio en el curso)
PUT	/api/laboratorios/:id	Actualizar laboratorio
DELETE	/api/laboratorios/:id	Eliminar (desactiva flag del curso si no hay más)

5.6 Secciones — /api/secciones

Método	Ruta	Descripción
GET	/api/secciones	Listar secciones

Método	Ruta	Descripción
GET	/api/secciones/:id	Obtener por ID
GET	/api/secciones/ curso/:cursoId	Secciones de un curso
POST	/api/secciones	Crear sección (requiere curso_id, letra)
PUT	/api/secciones/:id	Actualizar sección
DELETE	/api/secciones/:id	Eliminar sección

5.7 Sección Laboratorio — /api/seccion-laboratorio

Método	Ruta	Descripción
GET	/api/seccion-laboratorio	Listar todas
GET	/api/seccion-laboratorio/:id	Obtener por ID
POST	/api/seccion-laboratorio	Asignar lab a sección (seccion_id, laboratorio_id)
PUT	/api/seccion-laboratorio/:id	Actualizar asignación
DELETE	/api/seccion-laboratorio/:id	Eliminar asignación

5.8 Docente-Curso — /api/docente-curso

Método	Ruta	Descripción
GET	/api/docente-curso	Listar relaciones
POST	/api/docente-curso	Crear relación (docente_id, curso_id, puede_laboratorio)
DELETE	/api/docente-curso/:id	Eliminar relación

5.9 Configuración — /api/configuracion

Método	Ruta	Descripción
GET	/api/configuracion	Obtener configuración actual del AG
PUT	/api/configuracion	Actualizar parámetros. Si cambia rango de horas, regenera periodos automáticamente

5.10 Periodos — /api/periodos

Método	Ruta	Descripción
GET	/api/periodos	Listar periodos generados. Si no existen, los genera automáticamente desde la configuración.

5.11 Días Horario — /api/dias-horario

Método	Ruta	Descripción
GET	/api/dias-horario	Listar agrupaciones de días con sus días asociados (incluye JOIN con dias_horario_dia)

5.12 Algoritmo — /api/algoritmo

Método	Ruta	Descripción
POST	/api/algoritmo/ejecutar	Ejecutar el AG. Body opcional: nombre, tamaño_poblacion, max_generaciones, metodo_seleccion, metodo_cruce, metodo_mutacion, tasa_mutacion, aptitud_objetivo
GET	/api/algoritmo/estado	Estado actual de la ejecución en curso (generación, mejor aptitud, conflictos)
GET	/api/algoritmo/historial/:id	Historial de aptitud por generación del horario :id
GET	/api/algoritmo/configuracion	Configuración del AG (alias de /api/configuracion)
PUT	/api/algoritmo/configuracion	Actualizar configuración del AG
GET	/api/algoritmo/test-cromosoma	Test: generar población y mostrar estructura de genes
GET	/api/algoritmo/test-aptitud	Test: evaluar aptitud de un individuo aleatorio
GET	/api/algoritmo/test-seleccion	Test: comparar métodos de selección
GET	/api/algoritmo/test-cruce	Test: comparar métodos de cruce
GET	/api/algoritmo/test-mutacion	Test: comparar métodos de mutación
GET	/api/algoritmo/test-algoritmo	Test: ejecutar AG completo y devolver resultado

5.13 Horarios — /api/horarios

Método	Ruta	Descripción
GET	/api/horarios	Listar horarios generados
GET	/api/horarios/:id	Obtener horario completo (grilla de asignaciones)
GET	/api/horarios/:id/conflictos	Conflictos detectados en el horario

Método	Ruta	Descripción
GET	/api/horarios/:id/reporte	Estadísticas del horario (aptitud, generaciones, tiempo)
GET	/api/horarios/:id/exportar/csv	Exportar horario como archivo CSV
PUT	/api/horarios/:id/detalle/:detalleId	Editar manualmente una asignación del horario
PUT	/api/horarios/:id/activar	Marcar este horario como el activo (desactiva los demás)
DELETE	/api/horarios/:id	Eliminar horario y su historial

5.14 Importaciones — /api/importar

Todos los endpoints de importación reciben multipart/form-data con el campo archivo (archivo CSV).

Método	Ruta	Entidad importada
POST	/api/importar/docentes	Docentes
POST	/api/importar/cursos	Cursos
POST	/api/importar/docente-curso	Relaciones docente-curso
POST	/api/importar/salones	Salones
POST	/api/importar/laboratorios	Laboratorios
POST	/api/importar/secciones	Secciones
POST	/api/importar/seccion-laboratorio	Asignaciones sección-laboratorio

5.15 Reset — /api/reset

Método	Ruta	Descripción
GET	/api/reset/estado	Conteo de registros en todas las tablas operativas
DELETE	/api/reset/completo	Elimina cursos, docentes, salones, secciones, horarios. Conserva días, días_horario y configuración.
DELETE	/api/reset/horarios	Elimina solo los horarios generados y su historial

5.16 Manejo de Errores

La API utiliza una clase ApiError centralizada y un middleware global de errores. Los errores de PostgreSQL se mapean automáticamente a respuestas HTTP semánticas.

Código PG	HTTP	Descripción
22P02	400	Formato de dato inválido
23502	400	Campo obligatorio nulo
23503	409	Violación de integridad referencial
23505	409	Registro duplicado (UNIQUE constraint)
—	404	Recurso no encontrado
—	409	Ejecución del AG ya en curso
—	500	Error interno del servidor

6. Frontend — Vistas y Componentes

6.1 Tecnologías del Frontend

Tecnología	Uso
Vue 3 (Composition API)	Framework reactivo principal. Todas las vistas usan <script setup>.
Pinia	Gestión de estado global. Stores para datos compartidos entre vistas.
Vue Router	Enrutamiento SPA. Cada vista tiene una ruta definida.
Vite	Bundler y servidor de desarrollo con HMR.
CSS (scoped)	Estilos encapsulados por componente con variables CSS globales.

6.2 Mapa de Rutas

Ruta	Vista	Descripción
/	HomeView	Dashboard principal con accesos rápidos
/carreras	CarrerasView	CRUD de carreras académicas
/cursos	CursosView	CRUD de cursos con acciones masivas y filtros
/laboratorios	LaboratoriosView	CRUD de laboratorios asociados a cursos
/docentes	DocentesView	CRUD de docentes con horarios laborales
/salones	SalonesView	CRUD de salones y laboratorios físicos
/secciones	SeccionesView	Gestión de secciones por curso
/secciones-laboratorio	SeccionesLaboratorioView	Asignación de laboratorio a secciones
/docente-curso	DocenteCursoView	Relación docentes ↔ cursos
/dias-horario	DiasHorarioView	Vista de agrupaciones de días
/periodos	PeriodosView	Vista de periodos generados
/importaciones	ImportacionesView	Carga masiva CSV (individual y masiva)
/configuracion	ConfiguracionAgenteView	Parámetros del AG
/generacion	GeneracionView	Ejecución del AG y gestión de horarios
/reportes	ReportesView	Análisis de horarios y estadísticas
/mantenimiento	MantenimientoView	Reset y estado de la base de datos
/acerca-de	AcercaDeView	Información del sistema

6.3 Descripción Detallada de Vistas

HomeView — Dashboard

Vista de bienvenida que muestra el nombre del sistema y tarjetas de acceso rápido a los módulos principales. Implementa un menú desplegable para la sección de Cursos (Gestionar Cursos / Gestionar Laboratorios).

CarrerasView — Gestión de Carreras

CRUD completo de carreras académicas. Muestra una tabla con ID, código y nombre. Permite crear y editar mediante un modal con campos nombre y código. Confirma la eliminación antes de ejecutarla.

CursosView — Gestión de Cursos

Vista más completa de los catálogos. Incluye:

- Filtros por nombre/código de curso y por docente asignado.
- Acciones masivas: desactivar semestres pares, impares, todos; activar todos (con confirmación de seguridad).
- Tabla con columnas: ID, código, nombre, semestre, tipo, carrera, estudiantes, jornada, laboratorio, activo.
- Modal de creación/edición con todos los campos del curso.

DocentesView — Gestión de Docentes

CRUD de docentes. La tabla muestra registro personal, nombre, hora de entrada, hora de salida y estado activo. El formulario incluye validación de horarios (`hora_entrada < hora_salida`).

SalonesView — Gestión de Salones

CRUD de salones. Muestra nombre, capacidad, tipo (Teórico/Laboratorio), si el laboratorio está habilitado para clases teóricas, jornada disponible (mañana/tarde/ambas) y estado activo.

LaboratoriosView — Gestión de Laboratorios

CRUD de laboratorios. Cada laboratorio está vinculado a un curso. Incluye número de periodos del laboratorio y disponibilidad por jornada. Al crear o eliminar, actualiza automáticamente el flag `tiene_laboratorio` del curso.

SeccionesView — Gestión de Secciones

CRUD de secciones. Permite asignar salón fijo, docente fijo, periodo fijo y día de horario fijo. Estas fijaciones garantizan que el AG respete restricciones específicas.

SeccionesLaboratorioView

Asigna laboratorios a secciones. Cada sección puede tener un laboratorio asociado con salón fijo y docente fijo opcionales.

DocenteCursoView

Gestiona qué docentes pueden impartir qué cursos y si pueden hacerlo en laboratorio. Impide duplicados (mismo docente-curso).

ImportacionesView — Importación CSV

Vista con dos pestañas:

- Importación individual: selecciona el tipo de entidad, muestra el formato CSV esperado con ejemplo descargable, permite cargar el archivo y procesa fila a fila reportando éxitos y errores.
- Carga masiva: carga todos los tipos de entidad en secuencia automática (el orden garantiza integridad referencial).

ConfiguracionAgenteView

Muestra la configuración actual del AG en una tabla de parámetros. Un botón abre un modal para editar todos los parámetros: tamaño de población, máximo de generaciones, aptitud objetivo, tasa de mutación, métodos de selección/cruce/mutación, duración del periodo y rangos horarios.

GeneracionView — Ejecución del AG

Vista principal del flujo de trabajo. Tiene dos pestañas:

- Ejecución: formulario para configurar y lanzar el AG (nombre, parámetros con valores por defecto). Muestra progreso en tiempo real (generación actual, mejor aptitud). Feedback visual durante la ejecución.
- Horarios: tabla de horarios generados con opciones para ver detalle, activar como oficial y eliminar.

ReportesView — Análisis de Horarios

Vista avanzada de análisis. Incluye:

- Lista de horarios con filtros por ID, estado activo, método de selección, cruce y mutación.
- Vista de detalle de un horario: grilla completa de asignaciones.
- Reporte de conflictos detectados.
- Historial de evolución del AG (aptitud por generación).
- Exportación a CSV y PDF.

MantenimientoView

Herramientas de administración del sistema:

- Panel de estado: muestra el conteo de registros en cada tabla operativa con botón de refresco.
- Reset completo: elimina todos los datos operativos (con confirmación doble).
- Reset de horarios: elimina solo los horarios generados preservando los catálogos.

DiasHorarioView / PeriodosView

Vistas de solo lectura que muestran las agrupaciones de días y los periodos de tiempo generados automáticamente, respectivamente.

7. Instalación y Configuración

7.1 Requisitos Previos

Software	Versión mínima	Notas
Node.js	18.x LTS	Runtime del backend
npm	9.x	Gestor de paquetes (incluido con Node.js)
PostgreSQL	14.x	Motor de base de datos
Git	2.x	Control de versiones

7.2 Configuración de la Base de Datos

Paso 1. Crear la base de datos en PostgreSQL:

```
CREATE DATABASE genetic_scheduler;
```

Paso 2. Ejecutar el DDL del esquema:

```
psql -U postgres -d genetic_scheduler -f schedule_database_ddl.sql
```

El archivo DDL crea todas las tablas, índices, inserta los catálogos de días y horarios, y carga las 5 carreras de la División de Ingeniería del CUNOC.

7.3 Configuración del Backend

Paso 1. Instalar dependencias:

```
cd api
npm install
```

Paso 2. Crear el archivo .env en la raíz del backend:

```
PORT=3000
DATABASE_URL=postgresql://usuario:contraseña@localhost:5432/genetic_scheduler
```

Paso 3. Iniciar el servidor:

```
npm run dev # Modo desarrollo con nodemon
```

Verificar que el servidor esté funcionando:

```
GET http://localhost:3000/health
→ { "status": "ok", "message": "API disponible" }
```

7.4 Configuración del Frontend

Paso 1. Instalar dependencias:

```
cd frontend
npm install
```

Paso 2. Crear archivo .env en la raíz del frontend:

```
VITE_API_URL=http://localhost:3000/api
```

Paso 3. Iniciar el servidor de desarrollo:

```
npm run dev
```

La aplicación estará disponible en: <http://localhost:5173>

7.5 Flujo de Trabajo Recomendado

Siga este orden para cargar datos y generar un horario correctamente:

Pa so	Acción	Ruta/Módulo
1	Verificar carreras precargadas	GET /api/carreras
2	Importar o crear docentes	/importaciones o /docentes
3	Importar o crear salones	/importaciones o /salones
4	Importar o crear cursos	/importaciones o /cursos
5	Crear laboratorios para cursos que los tengan	/laboratorios
6	Crear secciones por curso	/secciones
7	Asignar laboratorios a secciones	/secciones-laboratorio
8	Asignar docentes a cursos	/docente-curso
9	Revisar/ajustar configuración del AG	/configuracion
10	Ejecutar el algoritmo genético	/generacion
11	Revisar reportes y activar horario	/reportes

8. Manejo de Errores y Validaciones

8.1 Capa de Errores del Backend

El sistema utiliza una arquitectura de errores centralizada implementada en `src/services/common/errors.js`:

- `ApiError`: clase que extiende `Error` con un campo `status HTTP`. Permite lanzar errores semánticos desde cualquier capa del sistema.
- `mapPgError`: función que intercepta errores de PostgreSQL (por código de error) y los convierte en `ApiError` con mensajes descriptivos en español.

El middleware global de Express (en `index.js`) captura todos los errores no manejados, registra en consola los errores 5xx y devuelve JSON estructurado al cliente.

8.2 Validaciones por Módulo

Módulo	Validaciones implementadas
Cursos	tipo debe ser "obligatorio" o "optativo". nombre único (case-insensitive). campos requeridos: nombre, codigo, semestre, tipo. No permite desactivar <code>tiene_laboratorio</code> si existe laboratorio asociado.
Laboratorios	<code>curso_id</code> debe existir. Solo un laboratorio por curso (UNIQUE). Al crear/eliminar actualiza <code>tiene_laboratorio</code> del curso.
Secciones	<code>curso_id</code> y letra requeridos. <code>curso_id</code> debe existir en BD.
Docente-Curso	<code>docente_id</code> y <code>curso_id</code> requeridos. Ambos deben existir. Combinación única (UNIQUE constraint).
Configuración	<code>validateConfigRanges</code> : <code>hora_inicio < hora_fin</code> para mañana y tarde. <code>duracion_periodo</code> debe ser entero > 0 .
Algoritmo AG	Solo una ejecución simultánea (estado global). Devuelve 409 si ya hay una en curso.
Importaciones	Solo se aceptan archivos CSV (por MIME type y extensión). Errores por fila reportados individualmente sin detener la importación.

8.3 Manejo de Transacciones

Los servicios que realizan múltiples operaciones de escritura usan transacciones PostgreSQL (BEGIN / COMMIT / ROLLBACK) para garantizar atomicidad:

- `configuracion.service.js`: actualización de configuración + regeneración de periodos.
- `cursos.service.js`: acciones masivas (desactivar/activar semestres).
- `laboratorios.service.js`: creación/actualización con sync del flag del curso.
- `periodos.service.js`: creación inicial de periodos si no existen.

9. Glosario

Término	Definición
Algoritmo Genético (AG)	Técnica de optimización inspirada en la evolución biológica. Mantiene una población de soluciones candidatas que evolucionan mediante selección, cruce y mutación.
Aptitud (fitness)	Valor numérico que mide la calidad de una solución (individuo). Mayor aptitud = mejor horario. Se calcula sumando penalizaciones y bonos.
Cromosoma / Individuo	Una solución completa del problema (un horario). Está compuesto por un arreglo de genes.
Gen	La unidad mínima del cromosoma. Representa la asignación de una sección (teórica o laboratorio) a un salón, docente, periodo y día.
Elitismo	Estrategia que conserva los mejores individuos de una generación a la siguiente sin modificarlos.
Selección por torneo	Elegir el mejor individuo de un subconjunto aleatorio de K individuos de la población.
Selección por ruleta	Selección probabilística donde los individuos con mayor aptitud tienen mayor probabilidad de ser elegidos.
Cruce (crossover)	Operador genético que combina genes de dos padres para producir dos hijos.
Mutación	Operador genético que altera aleatoriamente genes de un individuo con una tasa configurable.
Periodo	Franja de tiempo de duración fija (por defecto 50 min). Los periodos se generan automáticamente según la configuración de rangos horarios.
Sección	Grupo de estudiantes de un curso identificado por una letra (A, B, C...).
Días Horario	Agrupación de días: "Lunes, Miércoles y Viernes" (LMV) para cursos teóricos; "Martes y Jueves" para laboratorios.
Distribución de Lab	La forma en que se distribuyen los 3 periodos de un laboratorio entre martes y jueves (3+0, 2+1, 1+2, 0+3).
Fijaciones	Restricciones duras en secciones: salon_fijo_id, docente_fijo_id, periodo_fijo_inicio_id, dia_horario_fijo_id. El AG siempre las respeta.
SPA	Single Page Application. La interfaz frontend es una SPA que no recarga la página en la navegación.